

JPEG Compression

Exploiting human visual system



Original PNG - 12 MB



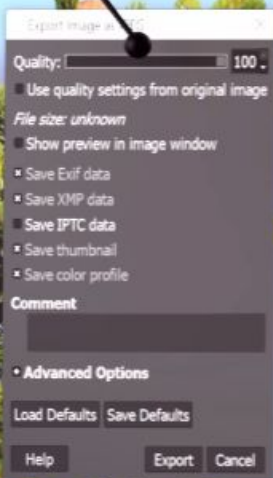
Compressed JPEG - 2.5 MB

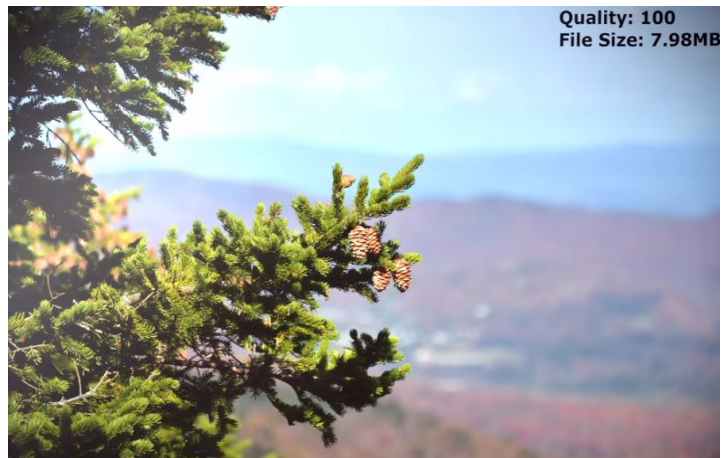
Motivation

- Human Visual System (HVS) perceive **luminance** change way better than we perceive a change in **color**.
- HVS is **less sensitive to high-frequency details (textures, edges)** than low-frequency (smooth) contents
- In JPEG Compression, we exploit this uniqueness of HVS.

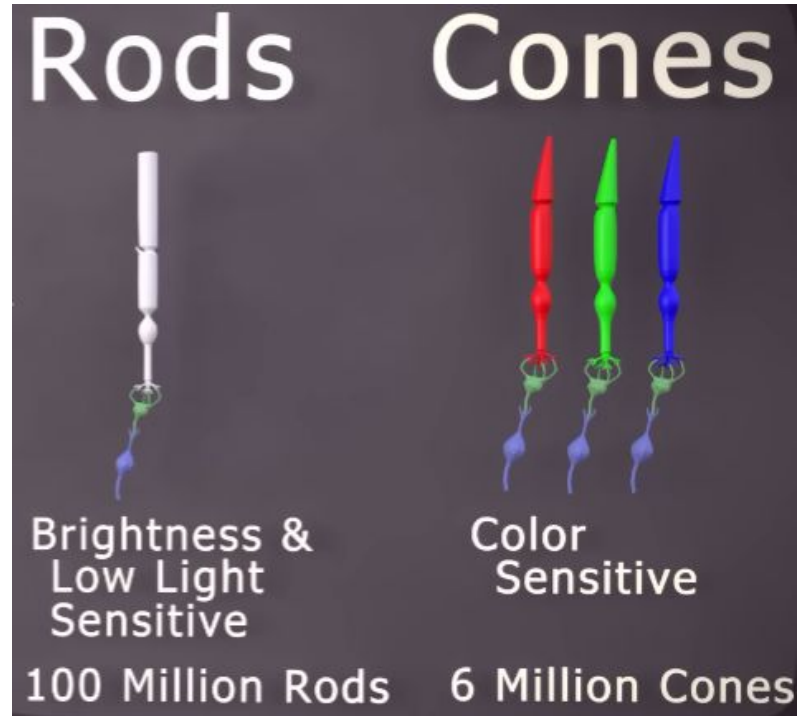


Quality





Why does this work?



Why does
JPEG
Work?



Rods



Brightness &
Low Light
Sensitive

100 Million Rods

Cones



Color
Sensitive

6 Million Cones

Luminance



Original



Chrominance



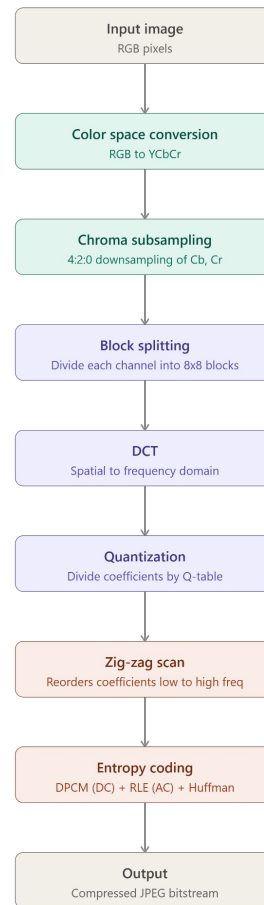
Branch.
Education



Branch.
Education

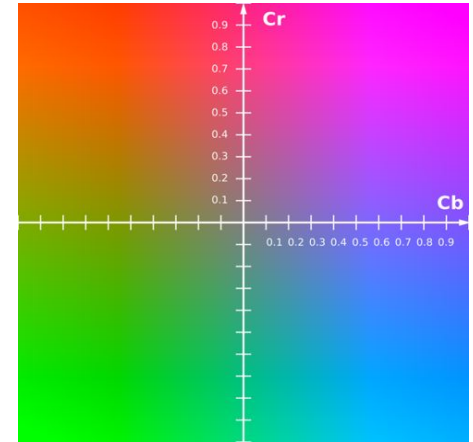
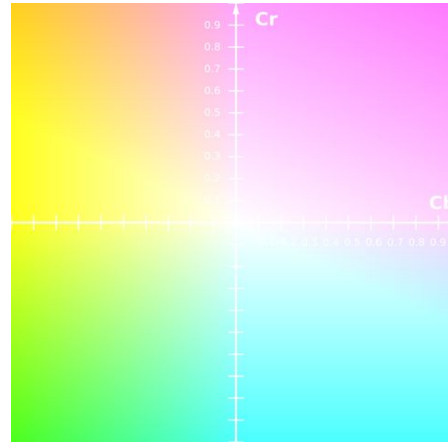
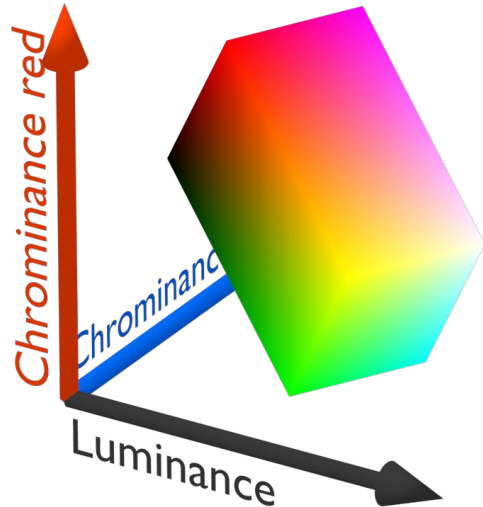
JPEG Compression Pipeline

1. Color Space Conversion to **YCbCr**
2. Chrominance Downsampling (Chroma Subsampling)
3. Block Splitting
4. Discrete Cosine Transformation (DCT)
5. Quantization
6. Zig-zag scan & Entropy Coding



Color Space Conversion to YCbCr

- We convert the image from RGB colorspace to YCbCr colorspace which allows us to separate Luminance information (Y) from Color Information.
- **Y** stands for **luma**, **Cb** stands for **blue difference chroma component** and **Cr** stands for **red difference chroma component**.



Color Space Conversion to YCbCr

- Conversion to YCbCr has different formulation equations depending on standards, the **BT.601** is used for JPEG compression:

$$\begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.168736 & -0.331264 & 0.5 \\ 0.5 & -0.418688 & -0.081312 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

This is essentially a linear transformation. The conversion is lossless except approximation errors. The inverse is given as –

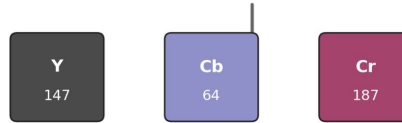
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1.402 \\ 1 & -0.344136 & -0.714136 \\ 1 & 1.772 & 0 \end{bmatrix} \cdot \begin{bmatrix} Y \\ C_b - 128 \\ C_r - 128 \end{bmatrix}$$

- In the forward conversion, notice how the **green** channel gets the highest weight in luma calculation. Its because humans can perceive the changes in green better than the other two.

RGB ↔ YCbCr conversion — worked example



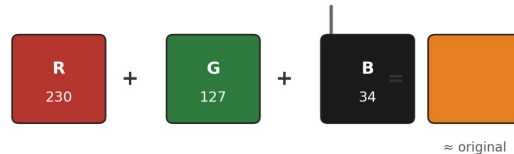
$$\begin{aligned} Y &= 0.299(230) + 0.587(126) + 0.114(34) = 146.61 \approx 147 \\ Cb &= -0.168736(230) - 0.331264(126) + 0.5(34) + 128 = 64.45 \approx 64 \\ Cr &= 0.5(230) - 0.418688(126) - 0.081312(34) + 128 = 187.48 \approx 187 \end{aligned}$$



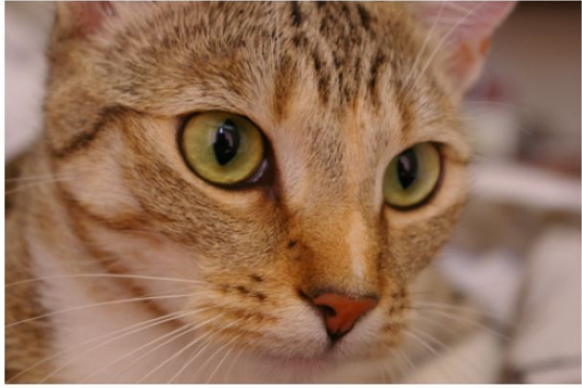
Inverse: YCbCr → RGB



$$\begin{aligned} R &= 147 + 1.402(187-128) = 147 + 82.72 = 229.72 \approx 230 \\ G &= 147 - 0.344136(64-128) - 0.714136(187-128) = 126.89 \approx 127 \\ B &= 147 + 1.772(64-128) = 147 - 113.41 = 33.59 \approx 34 \end{aligned}$$



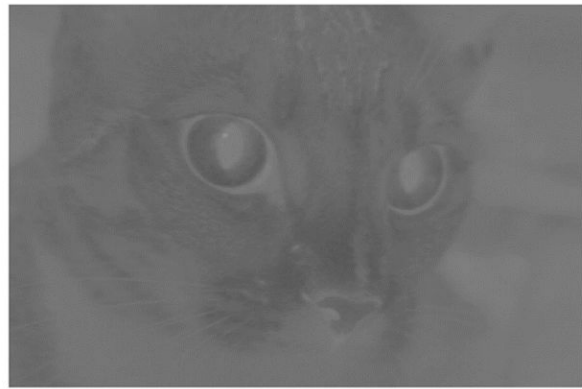
Reconstructed RGB(230, 127, 34) ≈ original RGB(230, 126, 34)
— conversion is lossless apart from rounding to integers



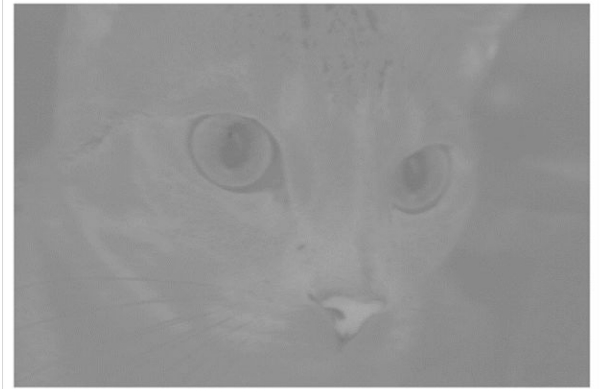
Lets convert our previously worked cat image to YCbCr color space. Notice how most of the information has been encapsulated in the Y channel. The other channels do not hold as much info.



Y channel



Cb channel

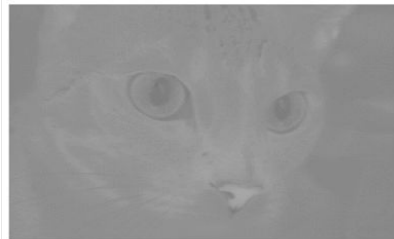
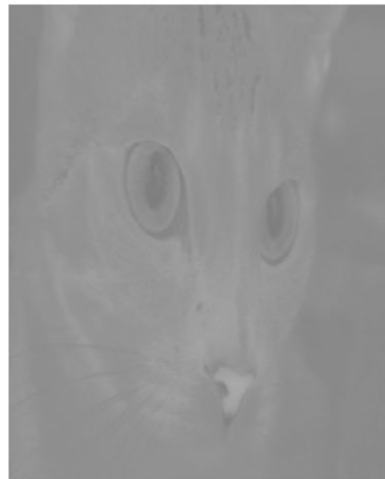
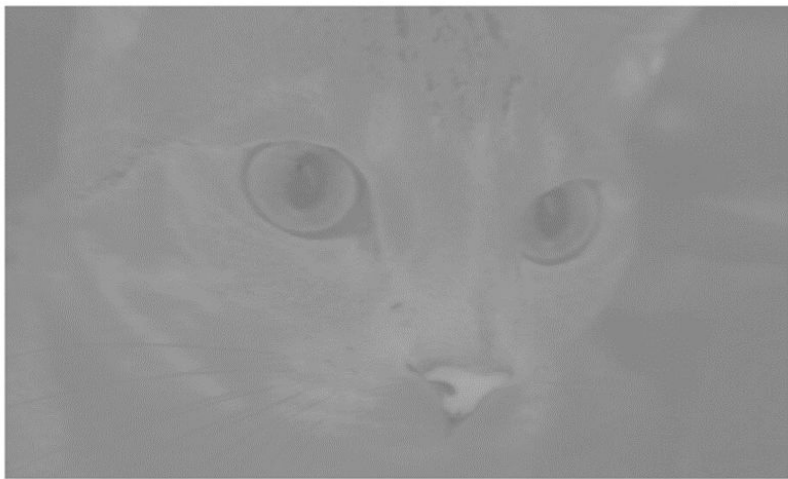


Cr channel

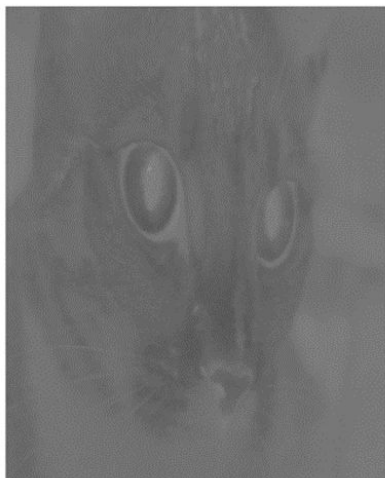
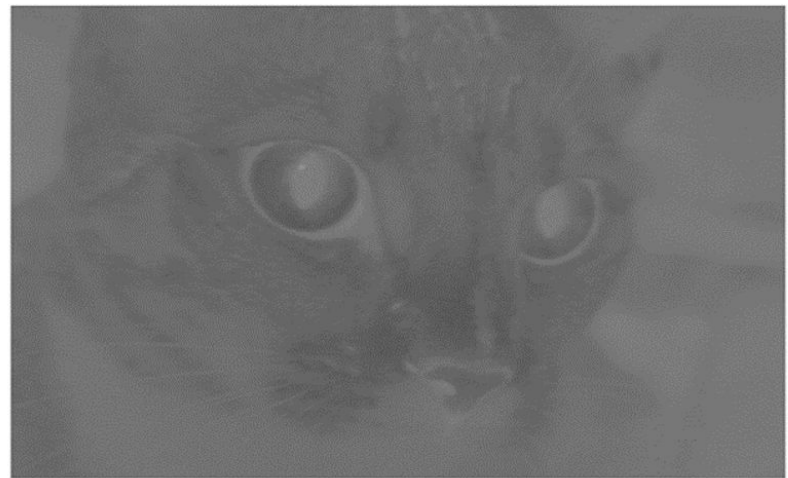
Chrominance Downsampling

- Since the **Cb** and **Cr** channels do not hold much information, we can down-sample and store the information in a much more compact form.
- This will result in information loss, but as established - we, humans, can not distinguish between colors that good.
- There are two widely adopted variants:
 - **4:2:2** downsampling where only “width” is downsampled by 2.
 - **4:2:0** downsampling where both “width” and “height” downsampled by 2.

Cr channel



Cb channel



Both 4:2:2 and 4:2:0 subsampling has been shown here for the Cb and Cr representation of the cat image.

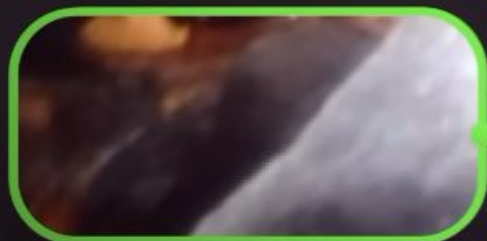
Chrominance Downsampling

- 4:2:0 compression is most commonly used for JPEGs. It reduces the storage requirements highly.
- Say you have an image of size $H \times W$. Then, **pixels** = $3HW$. After doing 4:2:0 subsampling, **pixels'** = $HW + HW/4 + HW/4 = 1.5 HW$. Essentially half.

JPEG

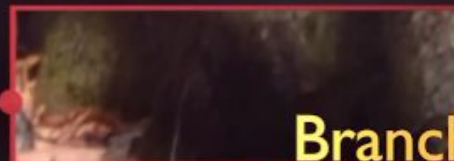
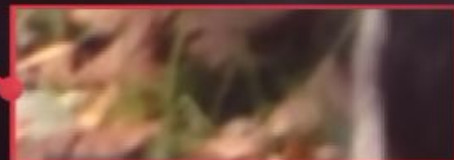
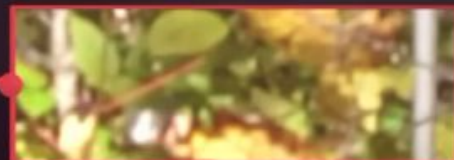
- 3. Discrete Cosine Transform
- 4. Quantization

Good at seeing



Bad at seeing

High Frequency Elements



Branch.
Education

Discrete Cosine Transformation

In close proximity (a patch) if sufficiently small (8x8), the image features do not change as much. Even if they do, it is very hard for humans to perceive such fast change in such a small locality. We can exploit this. Lets take the cat image, take a random 8x8 sample from the luma channel and extract the DCT weight patch.

Y Channel with 8x8 Patch Highlighted



8x8 Patch (Y channel)



Higher frequencies get super low weights compared to low frequencies, just like we expected. However, the zero frequency weight is very high. We can reduce this by shifting the intensities by -128. Essentially bringing the center intensity value to 0. The other weights do change too but not that much.

888.43	-17.77	-2.22	-1.17	-1.21	-0.70	0.80	-1.31
14.20	-1.02	0.20	0.95	0.69	0.15	1.00	-0.61
7.66	0.91	-0.32	-0.54	-1.65	-0.28	0.26	-0.69
0.65	0.24	0.91	-0.84	-1.03	-0.74	0.08	-0.11
0.92	-0.09	1.47	0.21	-0.77	0.41	0.42	-0.51
-0.06	-0.31	0.43	0.12	-1.13	0.24	0.08	0.08
-0.77	-0.12	-0.55	0.40	-0.26	0.53	0.23	-0.19
-0.51	-0.04	-0.08	-0.27	-0.46	0.40	-0.32	0.48

Discrete Cosine Transformation

888.43	-17.77	-2.22	-1.17	-1.21	-0.70	0.80	-1.31
14.20	-1.02	0.20	0.95	0.69	0.15	1.00	-0.61
7.66	0.91	-0.32	-0.54	-1.65	-0.28	0.26	-0.69
0.65	0.24	0.91	-0.84	-1.03	-0.74	0.08	-0.11
0.92	-0.09	1.47	0.21	-0.77	0.41	0.42	-0.51
-0.06	-0.31	0.43	0.12	-1.13	0.24	0.08	0.08
-0.77	-0.12	-0.55	0.40	-0.26	0.53	0.23	-0.19
-0.51	-0.04	-0.08	-0.27	-0.46	0.40	-0.32	0.48

Before shifting by 128

-135.57	-17.77	-2.22	-1.17	-1.21	-0.70	0.80	-1.31
14.20	-1.02	0.20	0.95	0.69	0.15	1.00	-0.61
7.66	0.91	-0.32	-0.54	-1.65	-0.28	0.26	-0.69
0.65	0.24	0.91	-0.84	-1.03	-0.74	0.08	-0.11
0.92	-0.09	1.47	0.21	-0.77	0.41	0.42	-0.51
-0.06	-0.31	0.43	0.12	-1.13	0.24	0.08	0.08
-0.77	-0.12	-0.55	0.40	-0.26	0.53	0.23	-0.19
-0.51	-0.04	-0.08	-0.27	-0.46	0.40	-0.32	0.48

After shifting by 128, $P(x, y) \leftarrow P(x, y) - 128$

Quantization

Now since, the high frequencies don't matter as much to us. We can actually suppress these weights. We suppress these weights by dividing them by a certain scaling factor (provided below as the Quantization of Q table). These scaling factors can be more or less aggressive depending on how much quality you want to retain. If you want to retain less quality, the Q factors will be bigger to increase weight suppression and vice versa.

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Q table (for Y channel) (50% quality)

-8.47	-1.62	-0.22	-0.07	-0.05	-0.02	0.02	-0.02
1.18	-0.09	0.01	0.05	0.03	0.00	0.02	-0.01
0.55	0.07	-0.02	-0.02	-0.04	0.00	0.00	-0.01
0.05	0.01	0.04	-0.03	-0.02	-0.01	0.00	0.00
0.05	0.00	0.04	0.00	-0.01	0.00	0.00	-0.01
0.00	-0.01	0.01	0.00	-0.01	0.00	0.00	0.00
-0.02	0.00	-0.01	0.00	0.00	0.00	0.00	0.00
-0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Scaled weights

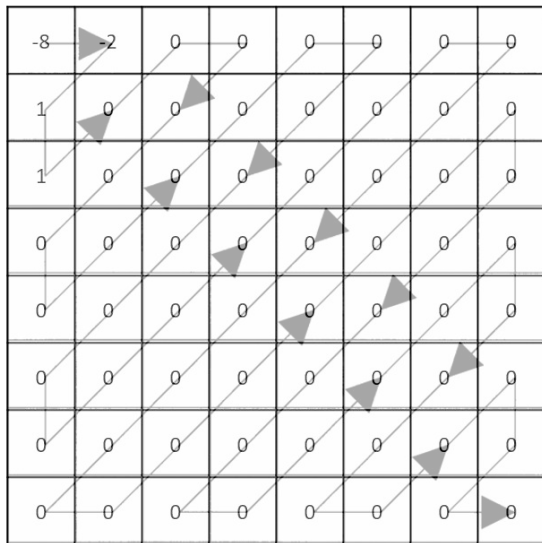
-8	-2	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Scaled weights (Rounded to integer)

Zig-zag scan & Entropy Coding

Now, you may ask, what is the point of doing this? It didn't save us any space. The representation dimensions are the same. You are right, we didn't save anything so far. But take a look at the rounded weight matrix after scaling. It's a sparse matrix. Sparse matrix can be stored quite efficiently. The exact method used in JPEG is as follows –

At first we read the data in sequence of ascending frequency (in a zig-zag order)



The data when read like this becomes, for this specific patch –

$$[-8, -2, 1, 1, 0, \dots, 0] = \{-8, -2, 1[\times 2], 0[\times 61]\}$$

So there are 61 0s in total. This will be the case most of the time. There will be a lot of 0s and we encode this data as run-length encoding (RLE) to save space. The bit representation is stored using Huffman encoding to save up even more space when representing in binary. Both RLE and Huffman encoding are examples of entropy encoding.

The exact representation is a bit complicated and you can read more about it from [here](#).

Zig-zag scan & Entropy Coding

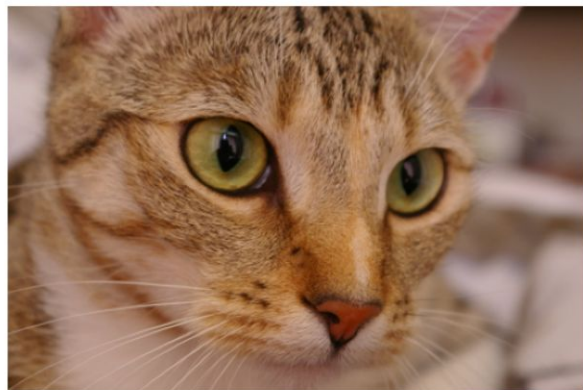
We do a similar quantization step with Cr and Cb and we also store them in similar ways. However, we suppress the Cr and Cb channels more than the Y channel. The Q table for Cr and Cb channels for 50% quality retention is –

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

In case the Cr and Cb has been subsampled, an 8x8 patch accounts for 8x16 original pixels for 4:2:2 subsampling and 16x16 original pixels for 4:2:0 subsampling.

Output

When everything is said and done, JPEG conversion on our cat image –



Original image



JPEG image

A little bit patchy if you look too hard.

The raw image requires 405900 bytes whereas the JPEG version requires 13773 bytes which is around 30 times compression. And yet, they look the same. I used 50% JPEG quality for the conversion.